Available at
www.ElsevierComputerScience.com
POWERED BY SCIENCE DIRECT
Theoretical Computer Science 310 (2004) 117–134

Theoretical Computer Science

www.elsevier.com/locate/tcs

Minimal schedulability interval for real-time systems of periodic tasks with offsets

Annie Choquet-Geniet* , Emmanuel Grolleau

Laboratoire d'Informatique Scientifique et Industrielle (LISI), Université de Poitiers & E.N.S.M.A., BP 40109, Téléport 2, Site du Futuroscope, F-86961 Futuroscope Chasseneuil, Cédex, France

Received 12 November 2002; received in revised form 14 May 2003; accepted 2 June 2003
Communicated by D. Peleg

Abstract

We consider real-time systems in highly safety context where tasks have to meet strict deadlines. Tasks are periodic, may have offsets, share critical resources and be precedence constrained. Off-line scheduling should be of great help for such systems, but methods proposed in the literature cannot deal with them. Our aim is to extend and improve the well-known cyclicity result of Leung and Merill to every scheduling algorithm and to systems of interacting tasks with offsets. One of the main benefit of our result is to enable the use of off-line scheduling methods for those real-time critical systems.
© 2003 Elsevier B.V. All rights reserved.

Keywords: Real-time systems; Schedulability analysis; Idle task; Acyclic idle slots; Processor request diagram; Schedulability interval; Blocking chains

1. Introduction

1.1. The validation problem

We consider applications composed of interacting periodic tasks, submitted to hard temporal constraints [15]. The core problem regarding real-time applications is the temporal validation: the question is whether the application can meet all its temporal constraints, and how this can be performed. Answering this question consists in proving the feasibility by means of suitable analytical criteria or in defining a *feasible* schedule

* Corresponding author. Tel.: 33-5-49-49-80-68; fax: 33-5-49-49-80-64.
E-mail addresses: ageniet@ensma.fr (A. Choquet-Geniet), grolleau@ensma.fr (E. Grolleau).

0304-3975/$ - see front matter © 2003 Elsevier B.V. All rights reserved.
doi:10.1016/S0304-3975(03)00362-1

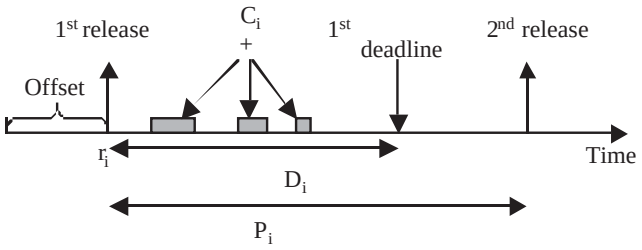


Fig. 1. Temporal parameters of a periodic real-time task $\tau_i = (r_i, C_i, D_i, P_i)$.

(which has to be infinite, since the application must run in steady state), i.e. a schedule which meets all its temporal and structural constraints. For the sake of studying the schedulability of real-time applications, we adopt the mostly considered temporal model [5,12,16]: each task τ_i is characterised by four temporal parameters: its *first release time or offset* r_i, its *worst-case computation time* C_i, its *relative deadline* D_i, which is the maximum delay tolerated between the release of an instance of the task and its completion, and its *period* P_i. In the following, we denote by r *the maximum offset*: $r = Max_{i=1,\ldots,n}\{r_i\}$ and P *the hyper period*: $P = \mathrm{LCM}$[1] $\{P_i,\ i = 1, \ldots, n\}$ (see Fig. 1).

The temporal validation of the application relies on the opportunity of choosing an appropriate scheduling strategy. Such a strategy can be defined as an on-line scheduling algorithm or be obtained through an off-line analysis.

1.1.1. On-line methods

The scheduling policy is implemented within the scheduler. On-line algorithms are usually priority driven: the processor is assigned to the pending task with the highest priority. The priority assignment can be either decided by the designer, or computed from the temporal parameters of the application (well-known strategies are priority assignment according to rate monotonic (RM), deadline monotonic (DM), earliest deadline first (EDF) and least laxity (LL) orders [10,11,12,14]). In restricted contexts, these algorithms are optimal in the sense that they answer the validation problem: if a feasible schedule exists, the schedule computed by the considered algorithm is feasible. Besides, if the tasks are *synchronous* (all the offsets are equal) and independent, EDF is optimal and the validation is performed by means of an analytical criterium, which computation is polynomial in the number of tasks. But if critical resources are used, the scheduling problem becomes NP-hard, and only sufficient feasibility conditions are still ahead [2,10,13]. Furthermore, in such context, there is no optimal on-line algorithm [13] and even for independent tasks systems with offsets, the validation problem is NP-hard [10].

[1] LCM denotes the least common multiple.

1.1.2. Off-line methods

A schedule, already computed, is stored within a table used by the dispatcher, avoiding the overhead caused by the execution of the scheduling algorithm. The main benefit comes from the raise of the scheduling power compared to on-line methods: on-line scheduling algorithms make decisions according to the instantaneous state of the system, meanwhile off-line methods are clairvoyant (they are based on a complete knowledge of the system). It follows that off-line methods are not reduced to *work-conserving* schedules (in a work-conserving schedule, a task never intentionally waits). This is mandatory in order for an algorithm to be optimal, since in some cases where critical resources are involved, the pending tasks have to wait in order to avoid a future blockage of an urgent task. Furthermore, off-line methods induce deterministic behaviours: since every action is planned within the schedule, the fluctuations of computation times do not affect the remainder of the schedule. Notice that such fluctuations may occur since we consider worst-case computation times. On-line strategies, on the opposite, can suffer of the instability problem: the decreasing of the computation time of a task of the system can lead some task of the application to lateness. Therefore, off line methods are of great help for validating highly coupled applications. The methods that can be found in the literature are usually exponential in time. Most of them rely on exhaustive branch-and-bound enumeration techniques [1,4,17,18]. They all consider restricted context. Either they deal with a finite set of aperiodic tasks, or, what is equivalent, a set of synchronous periodic tasks: only the set of instances released within the time interval $[0..\mathrm{PPCM}(P_i))$[2] has to be considered [10]. We prove in the present paper that these methods can also be applied to systems of periodic interacting tasks with offsets.

1.2. Our contribution

Two questions must be answered before any off-line methodology can be used to schedule such systems:

(1) Since only finite schedules can be computed and stored in the table used by the dispatcher, what is the (optimal) length of a feasible schedule that has to be stored?
(2) How can a steady state be got from this finite schedule?

These questions must be answered for any scheduling policy, since off-line methodologies propose most of the time an exhaustive computation of all the feasible schedules (see e.g. the method proposed in [9]). Our only assumption is that the scheduling policy is deterministic: if the system is in the same state at two different times, the scheduling decisions are the same.

Besides, answering these questions can help in performance study of schedules computed by on-line scheduling policies.

We define a *schedulability interval* as an interval $[0, a)$ such that the computation of a schedule on this interval provides enough informations for defining the infinite schedule. We prove the existence of such intervals, and show how to compute the shortest of them.

[2] $[a, b)$ is the set of values that are not less than a but less than b.

These problems have been studied by Leung and Merill [10] for the special case of EDF scheduling policy, and systems of independent tasks. They have shown that the time interval $[0, r+2*P)$ is a schedulability interval (not necessarily the shortest one), and that the schedule computed in the interval $[r+P, r+2*P)$ can be repeated infinitely often, defining a steady state. Goosens and Devillers [8] answer the cyclicity problem for systems of independent tasks with fixed priorities. They determine an upper bound for the time of entrance in the steady state.

The results presented in this paper extend and generalize these two results: we determine the *minimal length* of the schedule required for validation, for *any scheduling policy* (or, what is equivalent, the *earliest time at which the cyclic behaviour of the system begins*), and our results hold for systems *with offsets, using resources and precedence constraints*.

Our approach consists in proving the cyclicity of any deterministic feasible schedule, and in defining an algorithm which computes the time at which the cyclic behaviour begins. For that purpose, we pay special attention to the processor idleness, and we introduce the notions of cyclic and acyclic idle slots.

The paper is organised as follows: in Section 2, we define the idle task, the notion of acyclic idle slot and the processor request diagram, which helps to the location of the idle slots. In Section 3, we study systems of independent tasks with offsets. We first consider systems which fully load the processor, and work-conserving policy, and then, we get ride of both limitations considering systems with processor load less than 1 and non-work-conserving policies. Finally, in Section 4, we prove that the cyclicity result also holds for systems of interacting tasks.

2. Processor inactivity

2.1. The temporal model

We adopt the classical modelling of real-time tasks, as described in Fig. 1. We consider only systems of which the temporal parameters are a priori known. Such systems are the only ones for which the respect of the temporal parameters can be guaranteed. We suppose that $D_i \leqslant P_i$: an instance must be completed before the next one is released. We also suppose that the scheduling policy is deterministic, that is, the scheduling decisions depend only on the state of the system.

The processor activity is characterised by the *processor utilisation factor* $U = \sum_{i=1}^{n} C_i/P_i$. If $U > 1$, no feasible schedule can exist [5]. Now, if $U < 1$, the processor is not fully loaded, but remains idle $P(1-U)$ time units each hyper period. These idle slots are called *cyclic idle slots*, they are modelled by a periodic task τ_0, the *idle task*. Its temporal parameters are $C_0 = P(1-U)$, $D_0 = P_0 = P$. Its offset is discussed later.

2.2. Acyclic idle slots

The following example shows that, besides cyclic idle slots, some further idle slots may occur. Consider the task system $S_1 = \{\tau_1(0,1,4,4), \tau_2(1,3,6,6), \tau_3(3,1,4,4)\}$. Its

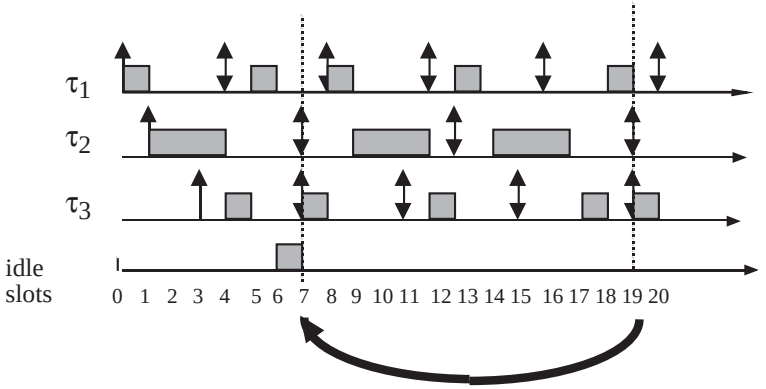


Fig. 2. EDF schedule for task system $S_1 = \{\tau_1(0,1,4,4), \tau_2(1,3,6,6), \tau_3(3,1,4,4)\}$.

processor utilisation factor is $U = \frac{1}{4} + \frac{3}{6} + \frac{1}{4} = 1$, the processor is fully loaded. Nevertheless, considering the schedule produced by the earliest deadline first policy[3] (see Fig. 2), we notice an idle slot at time 6. We can also notice that at time 7, the system adopts a cyclic behaviour (the cyclic part is delimited by the vertical dashed lines in Fig. 2). Finally, the length of the cyclic schedule is the hyper period $P = 12$. It follows that the (infinite) EDF schedule, denoted σ, can be expressed as $\sigma = \sigma_{[0...7)}\sigma^*_{[7...19)}$ where $\sigma_{[a...b)}$ is the restriction of σ to the time interval $[a...b)$, and $*$ the Kleene star [7]. The shortest schedulability interval is thus of length 19. This can be compared to the size of Leung and Merill's schedulability interval, which is of size 27.

The previous example points out the existence of some idle slots, that are not included within the idle task. They are called *acyclic idle slots*. Their existence can be deduced from an accurate study of the processor load.

2.3. The processor request diagram

In this section, we study the acyclic idle slots, which are closely related to the cyclicity problem. We assume that the system is scheduled by means of a *work-conserving policy*.[4] We assume furthermore that the tasks are independent. The request diagram represents the evolution of the cumulated requested processor load over the time (see Fig. 3). Let us introduce two functions: *Release*(t) is the sum of the computing times of the tasks that are released at time t; and *Request*(t) is the sum of the remaining computing times of the pending tasks at time t. The request diagram represents the function Request, which can be computed according to the following

[3] The priority of a task increases when its deadline becomes closer.

[4] A work-conserving policy never lets the processor idle when there are some pending tasks to process, e.g. EDF and RM scheduling policies are work-conserving.

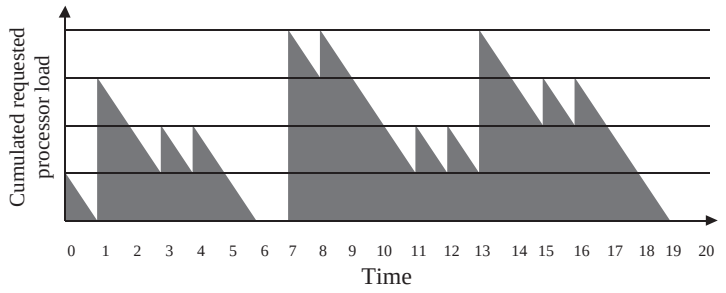


Fig. 3. Processor request diagram for system S_1.

rules:

$$\begin{aligned} Request(0) &= Release(0) \\ Request(t) &= if\ \ Request(t-1) > 0 \\ &\quad then\ Request(t-1) - 1 + Release(t) \\ &\quad else\ Release(t). \end{aligned}$$

- Each time unit, since the scheduling policy is work-conserving, one unit of request is processed, if there are pending tasks.
- Each time a task is released, the cumulated requested load is increased of its computation time.

An idle slot occurs only when the cumulated requested load is zero (see time 6 in Fig. 3), what does not depend on the choice of the work-conserving policy. In order to focus on the acyclic idle slots, we first consider fully loaded systems.

Lemma 1. *If the processor is fully loaded and if the tasks are independent, the number and the location of acyclic idle slots within schedules are independent of the chosen work-conserving policy.*

The proof is straightforward. Furthermore, the number of acyclic idle slots is bounded.

Lemma 2. *If the processor is fully loaded, the number n_c of acyclic idle slots in a feasible schedule is bounded.*

Proof. Let us consider a time interval $[0...r+NP)$, where N is an arbitrary positive integer. A task τ_i completes execution at least $\lfloor (r-r_i)/P_i \rfloor + (N*P)/P_i$ times within this interval (where $\lfloor x \rfloor$ is the greatest integer least or equal to x). Thus, the execution of task τ_i requires at least $(\lfloor (r-r_i)/P_i \rfloor + (N*P)/P_i) * C_i$ processing time units. It follows that the processor is busy at least during $\sum_{i=1}^{n} (\lfloor (r-r_i)/P_i \rfloor + (N*P)/P_i) * C_i = \sum_{i=1}^{n} \lfloor (r-r_i)/P_i \rfloor * C_i + N*P$ time units (remember that U is 1). There are thus at most $r + N*P - (\sum_{i=1}^{n} \lfloor (r-r_i)/P_i \rfloor * C_i + N*P) = r - \sum_{i=1}^{n} \lfloor (r-r_i)/P_i \rfloor$ time units

left for idle periods. Since it is independent of N, this number holds for the infinite schedule too. n_c is thus bounded. □

As a consequence, no acyclic idle slots occur within the steady state, which is characterised by a cyclic schedule, of period P. The acyclic idle slots come only from build-up of the system. In the next section, we go further into the study of the acyclic idle slots, for systems of independent tasks, and determine the start time of the periodic behaviour of the system. More precisely, we show that the last acyclic slot determines the end of the build-up phasis and the beginning of the steady state.

3. Independent task systems

3.1. Fully loaded task systems

3.1.1. The last acyclic idle slot

We focus on independent task systems and we establish the link between the time point of the last idle slot (it exists since the number of acyclic idle slots is bounded) and the time point of the beginning of the cyclic behaviour of the schedule. In the following, t_c denotes the time point of the last acyclic idle slot.

Proposition 3. *If the processor is fully loaded, the last acyclic idle slot occurs at the latest at* $r+P-1$.

Proof. The proof relies on an accurate study of the properties of the functions Release and Request. In the following, Δ denotes a natural integer. We need the three following results:

(a) *After r, the cumulated release requirement within any interval of size P is exactly* P : $\forall \Delta > 0$, $\sum_{t=0}^{P-1} Release(r+\Delta+t) = P$.

This results from $U=1$ and from the fact that within the time interval $[r+\Delta, r+\Delta+P)$, a task τ_i is released exactly P/P_i times.

Thus, we have $\sum_{t=0}^{P-1} Release(r+\Delta+t) = \sum_{i=1}^{n} P/P_i * C_i = P * U = P$.

(b) *After r, the cumulated processor request cannot decrease over an hyper period*: $\forall \Delta > 0$, $Request(r+\Delta+P) \geqslant Request(r+\Delta)$.

The proof is straightforward from the previous point. Between times $r+\Delta$ and $r+\Delta+P-1$, P processing time units have been requested by the released tasks, and at most P can have been processed. Furthermore, the released tasks are the same at times $r+\Delta$ and $r+\Delta+P$.

(c) *If an acyclic idle slot occurs after r, no acyclic idle slot can occur an hyper period later*: $\forall \Delta > 0$, $Request(r+\Delta) = 0 \Rightarrow Request(r+\Delta+P) > 0$.

Between $r+\Delta$ and $r+\Delta+P$, if an idle slot occurs at time $r+\Delta$, at most $P-1$ units of request can have been processed. Thus since between $r+\Delta$ and $r+\Delta+P$ the cumulated request due to releases of tasks is P, there is at least one remaining unit of request to process. The processor cannot then be idle, since we use a work-conserving policy.

Assume now that the last acyclic idle time occurs after $r+P-1$, that is to say, there is some natural integer Δ such that $t_c = r+P+\Delta$. We have $Request(r+P+\Delta)=0$. Using point b, we have $Request(r+\Delta)=0$ too, i.e. an idle slot occurs at time $r+\Delta$. But then, according to point c, no idle slot can occur at time $r+\Delta+P$, what contradicts our assumption.

3.1.2. The cyclicity result

If no acyclic idle slot occurs at all, we state $t_c=-1$, and the periodicity begins at time 0. We have the following proposition:

Proposition 4. *If the processor is fully loaded, the system is in the same state at times t_c+1 and t_c+P+1.*

Proof. At time t_c+1, since no requested processing time units were left by previously released tasks (else, no idle slot would have occurred at t_c), the cumulated requested processor load consists only of the released processor load:

$$Request(t_c+1) = Release(t_c+1). \tag{1}$$

After t_c, the cumulated processor request within any time interval of size P is equal to P processing time units, If it was less, an other idle time would occur, and it cannot be more, since the processor utilisation factor is $U=1$. We thus have $\sum_{i=1}^{P} Release(t_c+i+1)=P$. Furthermore, the cumulated requested processor load can never be zero, since no idle slot occurs after t_c. We have then, using the definition of function Request given in Section 2.3:

$$\begin{aligned}\sum_{i=1}^{P} Request(t_c+i+1) &= \sum_{i=1}^{P} [Release(t_c+i+1)+Request(t_c+i)-1] \\ &= P+\sum_{i=1}^{P} Request(t_c+i)-P \\ &= \sum_{i=1}^{P} Request(t_c+i).\end{aligned}$$

We deduce that

$$Request(t_c+P+1) = Request(t_c+1). \tag{2}$$

Furthermore, we have

$$Release(t_c+P+1) \geqslant Release(t_c+1). \tag{3}$$

All the tasks that are released at time t_c+1 are also released one hyper period later, but some tasks may have an offset greater than t_c. Finally, according to the definition of *Request*, we have

$$Request(t_c+P+1) \geqslant Release(t_c+P+1). \tag{4}$$

Combining Eqs. (1)–(4), we get

$$\begin{aligned} Release(t_c + P + 1) &\leqslant Request(t_c + P + 1) \\ &= Request(t_c + 1) \\ &= Release(t_c + 1) \\ &\leqslant Release(t_c + P + 1). \end{aligned}$$

It follows that $Request(t_c+P+1) = Release(t_c+P+1) = Request(t_c+1) = Release(t_c+1)$. This means that, at both time points, the only pending tasks are the tasks that have just been released, and these tasks are the same at $t_c + 1$ and $t_c + P + 1$. The system is therefore in the same state at both times.

Corollary 1. (1) *The minimal schedulability interval is the interval* $[0, t_c + P + 1)$.

(2) *The earliest cyclic part of the schedule begins at time* t_c, *and the schedule computed on the time interval* $[t_c + 1, t_c + P + 1)$ *defines the steady state of the system.*

(3) $t_c < r + P$, *and* t_c *can be deduced from the processor request diagram.*

(4) *If no acyclic idle slot occurs within the interval* $[0, P)$ *then* $t_c = -1$.

Proof. Point (1) results from the fact that no acyclic idle slot occurs within the cyclic part that cannot therefore start before $t_c + 1$ (2), and from Proposition 4. Point 2 comes from Proposition 4 and point (3) from Proposition 3.

In order to prove point (4), we first notice that in this case, $Release(0) = Release(P)$. Indeed, since no idle slot occurs, we have $\sum_{i=0}^{P-1} Release(i) = P$. We also have $Release(0) \leqslant Release(P)$. Thus $P \geqslant \sum_{i=1}^{P} Release(i) = \sum_{i=0}^{P-1} Release(i) + Release(P) - Release(0) = P + Release(P) - Release(0) \geqslant P$.

It comes $Release(P) - Release(0) = 0$. We then use the same arguments as for Proposition 4. □

3.2. Systems with processor utilization factor less than 1

We show in this section that we can in fact deal with *any scheduling policies, for any systems, either fully loaded or not*. Our proof relies on the introduction of the idle task within the set of the tasks to schedule.

We assume thus now that $U < 1$. As stated in Section 2, cyclic idle slots appear regularly and can be handled by the idle task τ_0. Scheduling the task system $S = (\tau_1, \ldots, \tau_n)$ is equivalent to scheduling the task system $S_0 = (\tau_0, \tau_1, \ldots, \tau_n)$ which fully loads the processor. We make no assumption about any priority which could be attached to the idle task. In this way, the idle task can possibly be scheduled, i.e. the processor may be idle, even if there are some other pending tasks. It follows that we can thus produce work-conserving as well as non-work-conserving policies. This is a strong requirement if we wish to establish properties for any schedule produced by off line strategies.

The results of Section 3.1 hold for the task system S_0, and can be extended to system S provided:

(a) we define the offset r_0 of the idle task,

(b) we can distinguish cyclic and acyclic idle slots, thus we can compute t_c.

The offset r_0 has an impact on t_c so, in order to minimize the size of the schedulability interval, we have to accurately choose r_0 in order to minimize t_c. The first idle slots have to be acyclic, and once the idle task has been released, no more acyclic idle slot may occur. We again use the processor request diagram. It points out the idle slots, we just have to determine which ones are acyclic and which ones cyclic. We proceed in the following way: we first consider the interval $[0, P)$. If there are only $P(1-U)$ idle slots, they are all cyclic, and, according to Corollary 1, we state $t_c = -1$ and $r_0 = 0$. Else, if n denotes the number of occurring idle slots, the $n - P(1-U)$ first idle slots are acyclic, and we consider next the time interval of length P starting after the last of these acyclic slots (occurring at t_{last}). We proceed in the same way: if only $P(1-U)$ idle slots occur, they are all cyclic, and we state $t_c = t_{\text{last}}$ and $r_0 = t_c + 1$. Else, the first idle slots are acyclic, and we go on moving the time interval until we reach an interval containing only $P(1-U)$ idle slots.

The algorithm is given in Appendix A and is illustrated in Fig. 4 and 5.

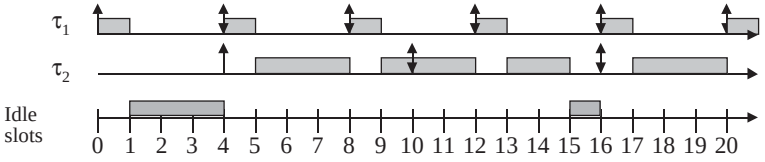


Fig. 4. RM schedule for the system $S_2 = \{\tau_1(0,1,4,4), \tau_2(4,4,6,6)\}$.

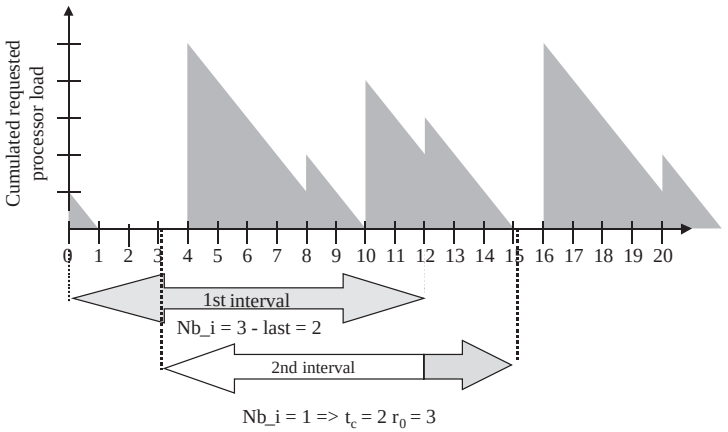


Fig. 5. The two first idle slots are acyclic, and the cyclic part starts at time 2. Only the coloured parts of the intervals have to be investigated.

Example. Consider the task system $S_2 = \{\tau_1(0,1,4,4), \tau_2(4,4,6,6)\}$.

We have $U = \frac{11}{12}$ thus $C_0 = 1$, $D_0 = P_0 = 12$. The system can e.g. be scheduled by the RM policy (see Fig. 4). The characteristics of the idle task are determined in Fig. 5 which shows the processor request diagram of system S_2. The algorithm considers first the interval $[0,12)$, it contains 3 idle slots, the two first ones are acyclic, and the second acyclic slot occurs at time 2. The second considered interval is then $[3,15)$ but only the part $[12,15)$ is further investigated. It contains only one idle slot, which is cyclic. The algorithm completes, and has computed $t_c = 2$ and $r_0 = 3$. The upper bound for t_c was $r + P = 16$, which is improved, since $t_c = 2$. The optimal schedulability interval is thus $[0,15)$ instead of $[0,28)$ using the upper bound. Only the computation of the diagram for the 15th first time units is required, the same holds for any schedule, and this is optimal. Note that the gain is still noticeable for on-line scheduling performance study, but that it can be huge for off-line methods that are often exponential in the length of the schedule.

4. System of interacting tasks

Off-line methodologies are particularly interesting for systems of interacting tasks. Therefore, we need to extend our results to such systems.

The use of critical resources, which must be used in mutual exclusion, has no influence on the results of Section 3: it just determines which of the pending tasks has to be processed, for a suitable resources management. Thus, the results established for independent tasks can be extended to task system using critical resources. The main difficulty comes from precedence constraints : some of the tasks cannot be executed in arbitrary order, but have to respect some *precedence relations* defined at the design step [3,6]. They represent synchronisation and communication relations. We note $\tau_i \rightarrow \tau_j$ (τ_i preceeds τ_j) if for any positive k, the kth instance of τ_i must have completed execution before the kth instance of τ_j can be processed. If $\tau_i \rightarrow \tau_j$, we further assume that $P_i = P_j$.

4.1. First example

Consider the task system $S_3 = \{\tau_1(0,1,4,4), \tau_2(1,1,4,4), \tau_3(2,1,4,4), \tau_4(0,1,4,4)\}$, with the precedence relation defined by $\tau_2 \rightarrow \tau_1$ and $\tau_1 \rightarrow \tau_4$. Using EDF policy, we get the schedule of Fig. 6. We have $U = 1$ thus there is no cyclic idle slot, but an acyclic idle slot occurs at time 0. Considering now the processor request diagram (see Fig. 7), we can see that the cumulated requested processor load is nevertheless positive at time 0. It follows that the previously used characterisation of idle slots does not hold anymore if precedence constraints are involved. In the case of system S_3, the idle slot comes from the fact that the already released task τ_1 waits for the not yet released task τ_2 and task τ_4, which is released too, waits for the completion of τ_1.

In precedence constrained systems, the cumulated processor load can be divided into two parts: the request of pending tasks, that can be processed immediately, and the request of waiting tasks, that cannot be processed at once. An idle slot occurs when there is no pending task: either all the previously released tasks have completed

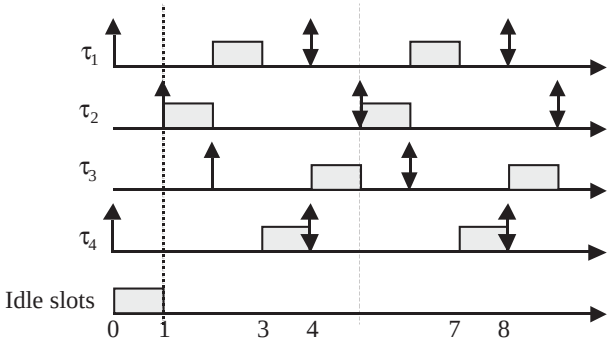


Fig. 6. EDF schedule for system $S_3 = \{\tau_1(0,1,4,4), \tau_2(1,1,4,4), \tau_3(2,1,4,4), \tau_4(0,1,4,4)\}$.

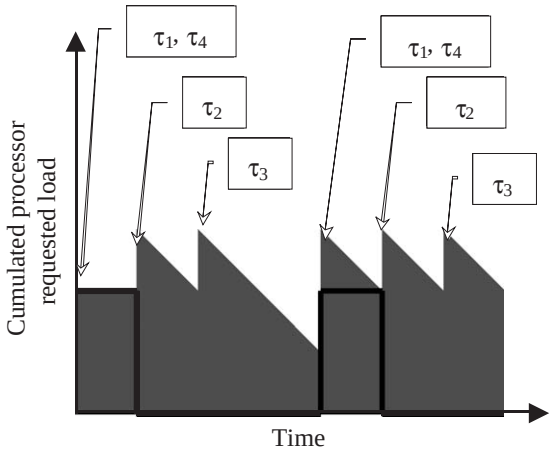


Fig. 7. The processor request diagram of system S_3. The newly released tasks are indicated.

execution; or all the released but not completed tasks are waiting for completion of another task, which is itself either not yet released, or waiting.

4.2. Blocking chains

The blocking chains express the cascade waitings induced by a not released instance of a task.

We call *t-blocking chain* a sequence $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$ such that the current released instance of τ_{i_k} waits for completion of the current instance of $\tau_{i_{k-1}}$,..., the current instance of τ_{i_3} waits for completion of the current instance of τ_{i_2}, the current released instance of τ_{i_2} waits for an instance of τ_{i_1} which has not yet been released. This can be formally expressed by:

- $\tau_{i_1} \to \tau_{i_2},\ \tau_{i_2} \to \tau_{i_3}, \ldots, \tau_{i_{k-1}} \to \tau_{i_k}$ (we thus have $P_{i_1} = P_{i_2} = \cdots = P_{i_k} = T$),
- $r_{i_2} \geqslant t, r_{i_3} \geqslant t, \ldots, r_{i_k} \geqslant t$,

- there is some $q(q = \lfloor (t - r_{i_2})/T \rfloor = \lfloor (t - r_{i_3})/T \rfloor = \lfloor (t - r_{i_k})/T \rfloor)$ such that the qth instances of tasks $\tau_{i_2}, \tau_{i_3}, \ldots, \tau_{i_k}$ have been released, but not the qth instance of task τ_{i_1}.

This last point can itself be expressed by the following inequalities:

$t \geqslant r_{i_2} + \lfloor (t - r_{i_2})/T \rfloor * T, \ldots, t \geqslant r_{i_k} + \lfloor (t - r_{i_k})/T \rfloor * T$ (where $r_{i_j} + \lfloor (t - r_{i_j})/T \rfloor * T$ is the release time of the current instance of task τ_{i_j}) and $t < r_{i_1} + \lfloor (t - r_{i_1})/T \rfloor * T$ (the qth instance of τ_{i_1} is not yet released).

Example. If we consider system S_2, the sequence τ_2, τ_1, τ_4 is a 0-blocking chain.

We define the *t-blocking set* as the set $B(t)$ of all the t-blocking chains.

An idle slot occurs at time t when the processor requested load comes only from tasks belonging to t-blocking chains.

The extension of the cyclicity result to precedence constrained systems relies on the cyclicity of t-blocking sets, which is stated in Proposition 5.

Proposition 5. *After r, the blocking sets are periodic, with period P.*

Proof. In the following, Δ denotes a natural integer.

- We first prove that a Δ-blocking chain is also a $(\Delta+P)$-blocking chain: $B(\Delta) \subseteq B(\Delta + P)$: Let $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$ be a Δ-blocking chain. At time Δ, the qth instances of tasks $\tau_{i_2}, \ldots, \tau_{i_k}$ have been released, with $q = \lfloor (\Delta - r_{i_2})/T \rfloor = \lfloor (\Delta - r_{i_3})/T \rfloor = \lfloor (\Delta - r_{i_k})/T \rfloor$, but not the qth instance of τ_{i_1}. An hyper period later, the $(q + P/T)$th instances of tasks $\tau_{i_2}, \tau_{i_3}, \ldots, \tau_{i_k}$ have been released, but not the $(q + P/T)$th instance of τ_{i_1}. Thus $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$ is a $(\Delta + P)$-blocking chain too.
- We then prove that a $(r + P + \Delta)$-blocking chain is also a $(r + \Delta)$-blocking chain: $B(r + \Delta + P) \subseteq B(r + \Delta)$.

  We first notice that at time $r + \Delta$, all tasks have been first released. We consider a $(r+\Delta+P)$-blocking chain $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$. The instances number $q = \lfloor (r+\Delta+P-r_{i_2})/T \rfloor$ of tasks $\tau_{i_2}, \ldots, \tau_{i_k}$ are released, but not these of task τ_{i_1}. For each task, the $(q-P/T)$th instance is released P time units sooner than the qth one, thus it is already released at time $r + \Delta$, for tasks $\tau_{i_2}, \ldots, \tau_{i_k}$, but not yet for task τ_{i_1}. Thus $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$ is also a $(r + \Delta)$-blocking chain.

Remark. It follows from this proof that after r, a blocking chain $\tau_{i_1}, \tau_{i_2}, \ldots, \tau_{i_k}$ is periodic of period T. Thus the whole set B admits P as period.

4.3. Idle slots and cyclicity result

We want again to determine an upper bound for the last acyclic idle slot, and prove that after it, the system enters its steady state.

We decompose the function Request in two functions : $Request = Pending + Waiting$.

- $Pending(t)$ is the sum of the remaining computation times of tasks that have been released before or at t, and that do not belong to a t-blocking chain (such tasks are the *pending tasks*).
- $Waiting(t)$ is the sum of the remaining computation times of tasks that have been released before or at t, and that belong to a t-blocking chain (such tasks are called *waiting tasks*).

An idle slot occurs when $pending(t)$ is zero, and thus $Request(t) = Waiting(t)$: the whole processor requested time comes from waiting tasks.

Since the releases of tasks and the blocking chains are periodic, with period P, the function Waiting is periodic, with period P after r.

We prove now that the acyclic idle slots, due to blocking chains, occur before $r+P$.

Proposition 6. *If the processor is fully loaded and the system precedence constrained, the last acyclic idle slot occurs at the latest at* $r + P - 1$.

Proof.

- We first prove that, *if there is no pending task at time* $r + \Delta$ (Δ *being a natural integer*), *there is at least one remaining pending task at time* $r + \Delta + P$.

  Indeed, after r, the cumulated release requirement on the interval $[r+\Delta, r+\Delta+P)$ is P, since $U = 1$. But only $P-1$ processor time units can have been processed, since $Pending(r + \Delta) = 0$ by assumption. Now, Waiting is periodic, thus $Waiting(r + \Delta) = Waiting(r + \Delta + P)$. Thus we have:

  $Request(r + \Delta + P) \geqslant Request(r + \Delta) + P - (P-1)$ (the term P corresponds to the cumulated released load and $(P-1)$ to the maximal amount of time units processed), i.e.

  $Request(r+\Delta+P) \geqslant Request(r+\Delta)+1$, and, because of the periodicity of Waiting, we have $Pending(r + \Delta + P) \geqslant Pending(r + \Delta) + 1 = 1$.
- We then prove that, *if there is at least one pending task at time* $r + \Delta$, *there is also at least one pending task at time* $r + \Delta + P$.

  We use the same arguments: here, the maximum number of processed time units is P, thus we have $Pending(r + \Delta + P) \geqslant Pending(r + \Delta) > 0$.

It follows that $Pending(r + \Delta + P)$ can never be equal to zero, thus no idle slot can occur after $r + P - 1$.

We can now extend the cyclicity result to systems of interacting tasks, the proof being the same as for independent tasks.

Proposition 7. *If the processor is fully loaded and the system precedence constrained, the system is in the same state at times* $t_c + 1$ *and* $t_c + P + 1$.

4.4. Non-fully loaded processor

If the processor utilization factor is less than one, we proceed in the same way as for systems of independent tasks, and we again get ride of the restriction to work-conserving policies. The only difference comes from the computation of t_c and r_0. We use the same algorithm as in Section 3.2, but we replace the test *If* $Req = 0$, which determines the occurrence of an idle slot (acyclic or cyclic) by the test *If* $Req = Waiting(t)$: an idle slot occurs when the whole requested load comes from the waiting tasks. We thus have to compute $Waiting(t)$. For that sake, we first compute the set $W(t)$ of the released blocked tasks (i.e. the set of the waiting tasks at time t).

We use a width first construction of the t-blocking chains. We first compute the set of released tasks directly blocked by other non-released tasks: τ_i belongs to $W_0(t)$ if there exists a task τ_j such that the current instance of τ_i waits for a non-released instance of τ_j:

- $\tau_i \rightarrow \tau_j$,
- $\lfloor (t - r_i)/P_i \rfloor < \lfloor (t - r_j)/P_j \rfloor$ (the number of the current instance of τ_i is less than the number of the current instance of τ_j).

Then we consider tasks that are blocked because they are waiting for completion of one of the tasks of W_0, and we go on until all the waiting tasks have been found. This is achieved by the algorithm given in Appendix B.

Example. consider system S_3. Since $U = 1$, there is no cyclic idle slot. In order to determine t_c, we first compute

- $Release(0) = 2$ (tasks τ_1 and τ_4 are released).
- $Request(0) = 2 = Release(0)$.
- Computation of W:
  - $W_0 = \{\tau_1\}$ (τ_1 is released, and waits for τ_2 which is not released),
  - $W = \{\tau_1, \tau_4\}$ (τ_4 waits for completion of τ_1 which is in W_0).
- $Waiting(0) = 2$.

There is thus an idle slot at time 0. The computation of Waiting on the interval $[1, 5]$ shows that no further idle slot takes place. The optimal schedulability interval is thus $[1, 5]$.

5. Conclusion

We have shown that systems of periodic interacting tasks with offsets behave cyclically after a build up phasis, and that the beginning of the steady state is independent of the scheduling policy. In a first time, we have paid special attention to the processor idleness. Two kinds of idleness have been observed: regular idleness, which can be deduced from the system load and occasional idleness. Regular idleness can be modelled by the (periodic) idle task. Its adjunction to other tasks leads to a processor utilisation factor equal to 100%. The main advantage of the introduction of the idle task τ_0 is that it enables to deal with any scheduling policies, work-conserving or not, for any systems, fully loaded or not. Notice further that this adjunction is only instrumental for the proofs, but that the user just has to focus on the processor diagram in order to determine the cycle of the tasks system. The remainder (occasional) idle slots are the acyclic idle slots. We have proven that:

- The number of acyclic idle slots is finite in any feasible schedule.
- The date of the last acyclic idle slot, denoted by t_c, is bounded by $r + P - 1$ and is independent of the scheduling policy.
- The system is in the same state at times $t_c + 1$ and $t_c + 1 + P$, what means that the schedule behaves cyclically after t_c.
- Furthermore, a suitable choice of the offset of the idle task leads to an optimal value for t_c.

It follows from these results that the schedule must be computed on the time interval $[0...t_c+P+1)$ and that the part of the schedule defined on the interval $[t_c+1, t_c+1+P)$ defines a steady state.

This result was mandatory in order to perform off-line schedulability analysis.

It enables the extension of methods that have been developed in literature for systems of aperiodic tasks or of periodic tasks without offsets. Since the schedulability interval is $[0, t_c+P+1)$, only the instances of tasks released within this interval have to be considered. Our results generalise those already known for systems of independent tasks with offsets scheduled by means of fixed priority algorithms, or by the earliest deadline policy. Furthermore, we have determined the optimal schedulability interval. The off line study can be performed together with the computation of this interval, and, although the schedulability problem remains NP_{hard}, we have here optimised its temporal complexity. This results have been used in [9] where an off line scheduling methodology based on a modelling by Petri nets of real-time systems has been developed. The schedulability of the application relied on the construction of the marking graph, whose depth could thus be bounded.

Appendix A. Algorithm of computation of t_c and r_0

```
Req:=  0;
idle_slots:= empty queue; -- contains the dates of the idle
slots occurring within the interval [tfirst, t]
nb_i:= 0; -- number of idle slots within the considered
interval, i.e. length of idle slots
tfirst :=  0; -- begining of the considered interval
t:= 0; -- end of the considered interval
last := -1; --date of the last known acyclic idle slot
found := false;

While (t < r + P) and not found DO
    Req:= Req + Release(t);
    IF Req = 0
       THEN --  There is an idle slot
          nb_i:= nb_i + 1;
          queue (idle_slots, t);
        ELSE Req:= Req - 1; -- the processor runs for one time unit
     END IF;
     IF t - tfirst + 1 = P --  the considered interval is of size P
     THEN
         IF nb_i > P(1 - U) -- some of the idle slots are acyclic
         THEN
            last := idle_slots[nb_i - P(1 - U)] -- the first idle
            slots of the interval are acyclic
         ELSE -- the interval contains no acyclic idle slots.  The last
```

```
                    acyclic idle slot has already been found
            found := true;
          END IF;
         tfirst := tfirst + 1; -- the begining of the interval is moved
                                  of one time unit
         IF head(idle_slots) = tfirst - 1 -- the first idle slot in
           the queue does no more belong to the considered interval, thus
           it must be removed from the queue
         THEN
            unqueue(idle_slots);
            nb_i := nb_i- 1;
         END IF;
      END IF;
      t := t + 1; -- the end of the interval is moved
      of one time unit
END DO;
t_c:= last;
r_0:= last + 1;
```

Note that the construction of the queue idle_slots can be achieved on the fly e.g. when simulating a scheduling algorithm.

Appendix B. Computation of the waiting tasks

We recall that W_0 is the set of the released tasks that are directly blocked by a non released task.

Remaining_tasks $:= \{\tau_1, \tau_2, \ldots, \tau_n\} \setminus W_0$ [5]

```
REPEAT
    W := W0;
    FOR each tasks τi in Remaining_task DO
         IF there is some task τj in W0 such that
         ⌊ (t − ri)/Ti ⌋ = ⌊ (t − rj)/Tj ⌋ and τj → τj
          THEN
          W0 := W0 ⋃ {τi};
          Remaining_tasks := Remaining_tasks \ {τi}
          END IF;
    END DO;
UNTIL W = W0;
```

The function $Waiting(t)$ is then defined by $Waiting(t) = \sum_{\tau_i \in W_0} C_i$.

[5] $A \setminus B$ is the set of the elements of A which do not belong to B.

References

[1] T.P. Baker, Z.S. Su, Sequencing with due-dates and early start times to minimize maximum tardiness, Nav. Res. Logist. Quat. 21 (1974) 171–176.
[2] S.K. Baruah, L.E. Rosier, R.R. Howell, Algorithms and complexity concerning the preemptive scheduling of periodic, real-time tasks on one processor, Real-Time Systems 2 (1990) 301–324.
[3] J. Blazewicz, Scheduling dependent tasks with different arrival times to meet deadlines, in: E. Gelenbe, H. Bellner (Eds.), Modelling and Performance Evaluation of Computer Systems, North-Holland, Amsterdam, 1976, pp. 57–65.
[4] P. Bratley, M. Florian, P. Robillard, Scheduling with earliest start and due date constraints on multiple machines, Nav. Res. Logist. Quat. 22 (1) (1975).
[5] G.C. Buttazzo, Hard Real-Time Computing Systems, Kluwer Academic Publishers, Dordrecht, 1997.
[6] F. Cottet, J.P. Babau, Off-line temporal analysis of hard real-time applications, in: Second IEEE Workshop on Real-Time Applications, 1994.
[7] S. Eilenberg, Automata Languages and Machines, Vol. A, Academic Press, New York, 1976.
[8] J. Goosens, R. Devillers, The non-optimality of the monotonic priority assignment for hard real-time offsets free systems, J. Real Time Systems 13 (2) (1997) 107–126.
[9] E. Grolleau, A. Choquet-Geniet, Off line computation of real time schedules by means of petri nets, J. Discrete Event Dyn. Systems 12 (2002) 311–333.
[10] J.Y.T. Leung, M.L. Merill, A note on preemptive scheduling of periodic real-time tasks, Inform. Process. Lett. 11 (3) (1980) 115–118.
[11] J. Leung, J. Whitehead, On the complexity of fixed-priority scheduling of periodic real-time tasks, Performance Evaluation 2 (1982) 237–250.
[12] C.L. Liu, J.W. Layland, Scheduling algorithms for multiprogramming in a hard real-time environment, J. ACM 20 (1) (1973) 46–61.
[13] A.K. Mok, Fundamental design problems for the hard real-time environments, Ph.D. Thesis, MIT, 1983.
[14] A.K. Mok, M.L. Dertouzos, Multi processor scheduling in a hard real-time environment, in: Proc. of Seventh Texas Conf. on Computer Systems, 1978.
[15] J.A. Stankovic, Misconception about real-time computing: a serious problem for the next generation systems, IEEE Compu. Magaz. 21 (10) (1988) 0–19.
[16] A. Stankovic, M. Spuri, K. Ramamritham, G.C. Buttazzo, Deadline Scheduling for Real-Time Systems, Kluwer Academic Press, Dodrecht, 1998.
[17] J. Xu, D.L. Parnas, Scheduling processes with release times, deadlines, precedence and exclusion relations, IEEE Trans. Software Eng. 16 (3) (1990) 360–369.
[18] J. Zamorano, A. Alonso, J.A. De La Puente, Automatic generation of cyclic schedules, in: WRTP'97—22nd IFAC/IFIP Workshop on Real Time Programming, 1997, pp. 145–151.